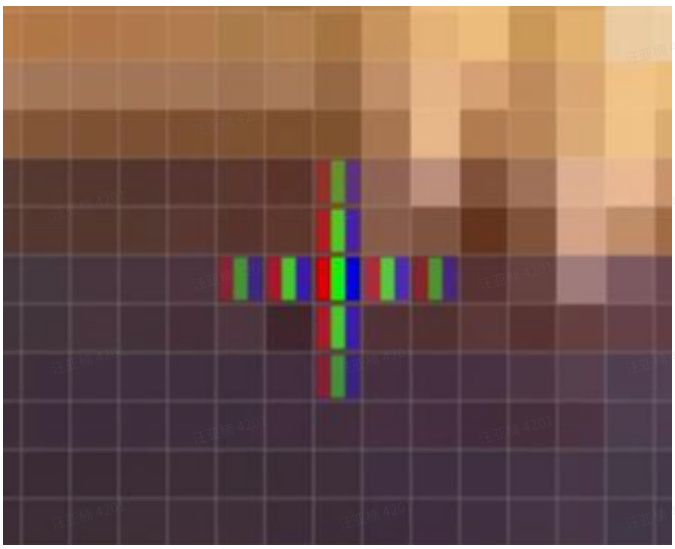
设备像素比

设备像素和CSS像素的比值，其中设备像素也叫物理像素，CSS像素又叫虚拟像素。

设备像素

从屏幕出厂的时候就决定了，具体就是屏幕上的发光点，这是硬件，无法更改的。

对于液晶屏幕而言，这些发光点也就是液晶体，就是通电后一个个会发光的物理设备，就像一个个极小的灯泡。它通过发出不同比例的红绿蓝光来组合成各种颜色光，最后组成图像。



CSS像素

CSS像素是一个虚拟的相对单位，用来表示一个图像相对不同设备中的统一尺寸。

要理解CSS像素的概念，就要理解“相对”的意思，我们来举个例子：

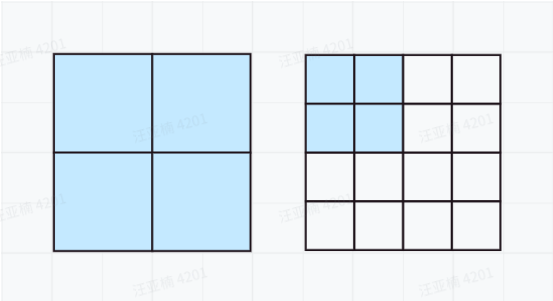
不同设备的物理像素的数量和大小（类似大灯泡和小灯泡）是不一样的，如果我们表示一张图像的大小要考虑上这些不同，那就可能会出现一种场景我们说这张图在A设备中是 `1px*1px` 的、但在B设备中是 `2px*2px` 的...但是这样会造成一个问题：图像的尺寸究竟是多大？

要解决这个问题，我们引入一种虚拟的像素单位，也就是CSS像素，用于描述一个图像**相对**不同设备中的统一尺寸，设备展示的大小由设备自身决定，但不管展示的大小相不相同，都表示同一个尺寸。

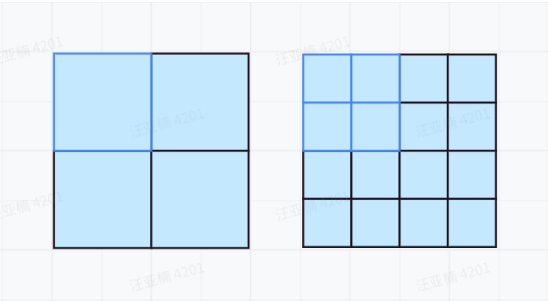
`px` 也就是CSS像素的单位。

设备像素比（device pixel ratio，缩写dpr）

假设有个 `2px*2px` 的视图在 `2*2` 物理像素尺寸的设备上完全匹配，如果这时将视图转移到一个 `4*4` 物理像素尺寸的设备上显示，按我们理解，它的显示应该只占用 `2*2` 的大小，如下图：



但是这意味视图无法占满设备，为了更好地展示，不再用原来的1个点（ `1*1` ）展示一个像素，而改用4个点（ `2*2` ）去展示一个像素，这时就能实现视图占满屏幕了。



如果我们要更好地理解设备像素比，那就要想清楚当前设备使用了多少实际物理像素去绘制单个CSS像素。我们上面说的用 `2*2` 物理像素来绘制 `1px*1px` 的CSS像素，这个比值 `2` 也就是设备像素比。

设备像素比=设备像素/CSS像素，先有“鸡”还是先有“蛋”？

有人会疑惑，看公式我们应该是通过设备像素和CSS像素去求设备像素比呀，但是CSS像素又是个相对单位，应该通过设备像素比才能算出1px在设备中实际需要参与绘制的物理像素点，这不就陷入一个逻辑怪圈了吗？

其实呢，对于浏览器来说，设备像素比是由屏幕本身和页面缩放值决定的，所以是先有设备像素比才能算出 `1px` 在设备中实际需要参与绘制的物理像素点。

页面缩放如何影响设备像素比

如果将浏览器页面放大一倍，假设原本 `1*1` 的物理像素能表示 `1px` ，但现在页面变大了， `1px` 看起来变大了，我们需要用 `2*2` 的物理像素去表示 `1px` ，所以设备像素值会从 `1` 变成 `2` 。

我们常在 `HTML` 中的 `meta` 标签中看到以下设置：

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

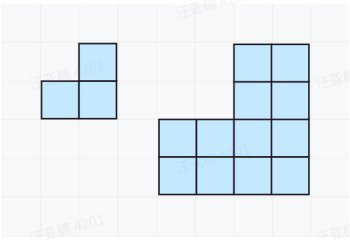
这里也就是将设备的物理像素宽度作为页面可视区域的CSS像素宽度，如果这时页面的设备像素比为2，且页面缩放为200%，那实际上**页面可视区域的CSS宽度=设备的物理像素宽度*设备像素比*缩放倍数**。

浏览器中的 `window.devicePixelRatio` 是已经考虑上页面缩放倍数的，也就是：`window.devicePixelRatio = 设备像素比*缩放倍数` 。

设备像素比在Canvas中的应用

为什么不考虑设备像素比，直接绘制Canvas会出现模糊?

原因在于画布的绘制是由我们自己控制的，如果我们没有考虑设备像素比，假设屏幕设备像素比为2，它会用4个物理像素点去绘制我们设置的 `1px` ，这也使得像素锯齿更加明显。



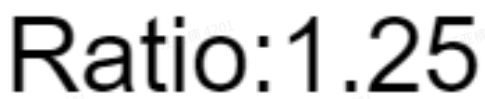
所以我们这时候需要自行处理，如果设备像素比为2，则画布绘制中所有的数值单位都乘以2，但这样会让画布尺寸变大一倍，这时我们再通过指定画布的 `style` 的宽高属性即可显示为正确尺寸。

```
...
<canvas id="_canvas"></canvas>
<script>
  const canvas = document.querySelector('#_canvas');
  const width = 200, height = 100;
  const dpr = window.devicePixelRatio;
  canvas.width = width * dpr;
  canvas.height = height * dpr;
  canvas.style.width = ${width}px;
  canvas.style.height = ${height}px;

  const ctx = canvas.getContext('2d');
  if (ctx) {
    ctx.font=${30 * dpr}px Arial
    ctx.fillStyle = '#000';
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
```

```
    ctx.fillText(Ratio:${dpr}, width / 2 * dpr, height / 2 * dpr);
  }
</script>
...
```

处理前



处理后

忽略设备本身的设备像素比，代码处理中设置越大越好吗？

当然不是，大图缩小展示会通过不同算法去压缩图像数据，这就像用十句话描述改用一句话去描述，后者肯定不如前者描述的清晰，至于压缩后的效果怎么样取决于选择什么样的压缩算法，但是浏览器的压缩算法在过度压缩后肉眼看就知道不咋地，所以最好的选择就是设置浏览器本身提供的设备像素比。